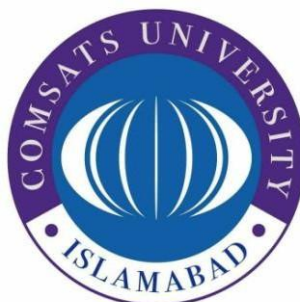


COMSATS University Islamabad

Attock Campus



Department Of Computer Science

<i>Course</i>	<i>Information Security Lab</i>
<i>Instructor</i>	<i>Mam Ambreen Gul</i>
<i>Assignment No.</i>	<i>04</i>

Student Details

<i>Registration No.</i>	<i>Name</i>
<i>FA24-BSE-017</i>	<i>Mubashir Hussain</i>

Task1:

Line-by-Line Explanation

1. **p** and **q** are small prime numbers chosen manually for simplicity.
2. **n = p × q** is the modulus used in RSA encryption and decryption.
3. **phi(n)** represents the number of integers less than n that are coprime to n.
4. **e** is the public key exponent chosen such that it is coprime with phi(n).
5. **find_d()** finds the private key exponent d such that
$$(e \times d) \bmod \phi(n) = 1$$
6. **Public key (e, n)** is used for encryption.
7. **Private key (d, n)** is used for decryption.
8. **encrypt()** converts each character to ASCII and applies RSA encryption formula:
$$c = m^e \bmod n$$
9. **decrypt()** applies RSA decryption formula:
$$m = c^d \bmod n$$
10. The decrypted ASCII values are converted back into characters.
11. The program is tested using a short string (name).

Security Analysis

This RSA implementation is designed **only for educational purposes**.

It is **not secure** for real-world use due to the following reasons:

- Very small prime numbers are used, making it vulnerable to brute-force attacks.
- No padding scheme (such as OAEP) is applied.
- Character-by-character encryption leaks patterns.
- Real RSA implementations use **large primes (2048 bits or more)**.

Therefore, this implementation should **never be used in real applications**.

```
1  # Step 1: Choose two small prime numbers
2  p=53
3  q=37
4  # Step 2: Compute n
5  n=p*q
6  # Step 3: Compute phi(n)
7  phi=(p-1)*(q-1)
8  # Step 4: Choose public key e (must be coprime with phi)
9  e=7
10 # Step 5: Find private key d (such that (e * d) % phi = 1)
11 def find_d(e,phi): 1 usage
12     for d in range(1,phi):
13         if (e*d)%phi==1:
14             return d
15 d = find_d(e,phi)
16 # Public and Private Keys
17 public_key=(e,n)
18 private_key=(d,n)
19 print("Public Key:",public_key)
20 print("Private Key:",private_key)
21 # ENCRYPT FUNCTION
22 def encrypt(message,public_key): 1 usage
23     e,n=public_key
24     encrypted = []
25     for char in message:
26         m=ord(char)          # Convert character to number
27         c=pow(m,e,n)         # RSA formula: c = m^e mod n
28         encrypted.append(c)
29     return encrypted
```

```

30 # DECRYPT FUNCTION
31 def decrypt(ciphertext,private_key): 1usage
32     d,n=private_key
33     decrypted=""
34     for c in ciphertext:
35         m=pow(c,d,n) # RSA formula: m = c^d mod n
36         decrypted+=chr(m) # Convert number back to character
37     return decrypted
38 message="Mubashir Hussain"
39 #Calling encrypt and decrypt function
40 encrypted_message=encrypt(message,public_key)
41 decrypted_message=decrypt(encrypted_message,private_key)
42 print("\nOriginal Message:",message)
43 print("Encrypted Message:",encrypted_message)
44 print("Decrypted Message:",decrypted_message)

```

Output:

```

Public Key: (7, 1961)
Private Key: (535, 1961)

Original Message: Mubashir Hussain
Encrypted Message: [1891, 290, 5, 1235, 1362, 1409, 635, 1373, 389, 1352, 290, 1362, 1362, 1235, 635, 961]
Decrypted Message: Mubashir Hussain

```

Task2:

Line-by-Line Explanation

Key Generation

1. **p** and **q** are small prime numbers chosen manually.
2. **n = p × q** is the modulus used in RSA operations.
3. **phi(n)** represents numbers coprime to n.
4. **e** is the public exponent chosen such that it is coprime with phi.
5. **find_d()** finds private key d such that

$$(e \times d) \bmod \phi = 1$$

6. **Public key (e, n)** is shared openly.
7. **Private key (d, n)** is kept secret.

Hashing

8. **create_hash()** uses SHA256 to convert the message into a fixed-length hash.
9. The hash is converted into an integer so RSA math can be applied.

Signing

10. **sign_message()** generates the message hash.
11. The hash is encrypted using the **private key**:

$$\text{signature} = \text{hash}^d \bmod n$$
12. This signature proves the sender's identity.

Verification

13. **verify_signature()** re-hashes the received message.
14. The signature is decrypted using the **public key**:

$$\text{hash} = \text{signature}^e \bmod n$$
15. If both hashes match, the message is authentic and unchanged.

Message Modification Test

16. A small change in the message alters the hash.
17. Verification fails, proving **integrity protection**.

Security Analysis

This digital signature implementation is **only for learning purposes** and is **not secure** for real-world use due to the following reasons:

- Very **small prime numbers** make RSA keys breakable.

- No secure padding scheme is used.
- Hash value is directly reduced modulo n.
- Character-level or small-number RSA is vulnerable to attacks.
- Real systems use **2048-bit or larger keys** and **secure padding**.

```

1  import hashlib # Library used for SHA256 hashing
2  # Step 1: Select two small prime numbers
3  p=17
4  q=11
5  # Step 2: Calculate modulus n
6  n=p*q
7  # Step 3: Calculate Euler's Totient Function
8  phi=(p-1)*(q-1)
9  # Step 4: Choose public exponent e (coprime with phi)
10 e=7
11 # Step 5: Function to calculate private key d
12 def find_d(e,phi): 1usage
13     # d must satisfy (e * d) mod phi = 1
14     for d in range(1,phi):
15         if (e*d)%phi==1:
16             return d
17 # Calculate private key
18 d = find_d(e,phi)
19 # Public and private key pairs
20 public_key=(e,n)
21 private_key=(d,n)
22 # Step 6: Create SHA256 hash of message
23 def create_hash(message): 2 usages
24     # Convert message to bytes and generate SHA256 hash
25     hash_object=hashlib.sha256(message.encode())
26     # Convert hexadecimal hash to integer
27     return int(hash_object.hexdigest(),16)
28
29 # Step 7: Sign message using private key
30 def sign_message(message,private_key): 1usage
31     d,n=private_key
32     # Generate hash of the message
33     message_hash=create_hash(message)
34     # Sign the hash using RSA formula
35     signature=pow(message_hash,d,n)
36     return signature
37
38 # Step 8: Verify signature using public key
39 def verify_signature(message,signature,public_key): 2 usages
40     e,n=public_key
41     # Generate hash of received message
42     message_hash=create_hash(message)
43     # Decrypt signature using public key
44     decrypted_hash=pow(signature,e,n)
45     # Compare hashes
46     return message_hash==decrypted_hash
47
48 # Step 9: Testing the Digital Signature
49 message="Mubashir Hussain"
50 # Sign original message
51 signature=sign_message(message,private_key)
52 print("Original Message:",message)
53 print("Signature:",signature)
54 # Verify original message
55 print("Verification (Original):",verify_signature(message,signature,public_key))
56 # Modify the message slightly
57 modified_message="Mmubashir Hussain"
58 # Verify modified message
59 print("Verification (Modified):",verify_signature(modified_message,signature,public_key))

```

Output:

```

Original Message: Mubashir Hussain
Signature: 87
Verification (Original): True
Verification (Modified): False

```